

FOREIGN KEY — внешний ключ и связи между таблицами

Пример связи между двумя

Рассмотрим пример связи «один ко многим» (1:N)

Одна запись в таблице `users` может иметь несколько

`users.id` (PK)

`users` Таблица пользователей

id	name	email	c
 1	Иван Петров	ivan@example.com	2
2	Анна Иванова	anna@example.com	2
3	Сергей Кузнецов	sergey@example.com	2
4	Елена Смирнова	elena@example.com	2
5	Дмитрий Волков	dmitry@example.com	2

Как это работает?

- 1 В таблице `users` у каждой записи есть уникальный идентификатор (первичный ключ, PK).
- 2 В таблице `orders` есть поле `user_id`, которое ссылается на

Зачем он нужен: аналогия из жизни

В студенческом билете есть номер группы, например "IT-21". Сам билет не хранит список всех студентов группы и расписание — он просто **ссылается** на группу по её номеру, а вся информация о группе лежит в другом месте (в деканате). Если группу переименоуют или расформируют, это одно изменение "в одном месте", а не в сотне билетов. Внешний ключ работает точно так же: столбец в одной таблице **ссылается** на первичный ключ строки в другой таблице, вместо того чтобы дублировать её данные.

Родительская и дочерняя таблица

В связи всегда есть:

- **родительская (главная) таблица** — та, на которую ссылаются (например, `courses` — курсы);
- **дочерняя (зависимая) таблица** — та, где лежит ссылка (например, `assignments` — задания, у каждого задания есть курс, к которому оно относится).

Синтаксис в SQLite

```
CREATE TABLE courses (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  title TEXT NOT NULL  
);
```

```
CREATE TABLE assignments (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  course_id INTEGER NOT NULL REFERENCES courses(id), -- внешний ключ на courses.id  
  title TEXT NOT NULL  
);
```

По умолчанию в SQLite проверка внешних ключей **выключена** — её нужно включить отдельно:

```
PRAGMA foreign_keys = ON;
```

После включения SQLite не даст вставить задание со ссылкой на несуществующий курс:

```
INSERT INTO assignments (course_id, title) VALUES (999, 'Тестовое задание');  
-- Error: FOREIGN KEY constraint failed (курса с id=999 не существует)
```

Три типа связей

Один к одному — редкий случай, когда каждой строке родительской таблицы соответствует ровно одна строка дочерней. Пример: один пользователь — один профиль с дополнительными настройками.

Один ко многим — самый частый тип связи. Одной строке родителя соответствует много строк потомка, но не наоборот. Примеры из этой же платформы:

- один курс (`courses`) — много заданий (`assignments`);
- одно задание (`assignments`) — много сдач (`submissions`), потому что разные студенты сдают одно и то же задание;
- один студент (`users`) — много попыток решения в SQL-тренажёре (`sql_attempts`).

```
SELECT assignments.title, submissions.student_id  
FROM submissions  
JOIN assignments ON submissions.assignment_id = assignments.id;
```

Многие ко многим — записи с обеих сторон могут соответствовать сразу нескольким записям с другой стороны. Классический пример: студенты и курсы — один студент учится на нескольких курсах, на одном курсе учится много студентов. Такую связь напрямую внешним ключом не выразить — нужна **промежуточная таблица** со своим составным ключом:

```
CREATE TABLE enrollments (  
  student_id INTEGER NOT NULL REFERENCES users(id),  
  course_id INTEGER NOT NULL REFERENCES courses(id),  
  PRIMARY KEY (student_id, course_id)  
);
```

Ссылочная целостность: что может пойти не так без внешних ключей

- **Висячая ссылка (аномалия удаления)** — удалили курс, а задания с этим `course_id` остались и указывают "в никуда".
- **Ссылка на несуществующее (аномалия вставки)** — создали задание с `course_id`, которого никогда не существовало (опечатка).
- **Рассинхронизация (аномалия обновления)** — если бы название курса дублировалось в каждой строке заданий вместо ссылки по `id`, при переименовании курса пришлось бы искать и менять его везде.

Внешний ключ с включённой проверкой (`PRAGMA foreign_keys = ON`) не даёт создать первую и вторую проблему на уровне базы данных — а хранение данных без дублирования решает третью.

Что делать при удалении родителя: ON DELETE

```
CREATE TABLE assignments (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  course_id INTEGER NOT NULL REFERENCES courses(id) ON DELETE CASCADE  
);
```

- **ON DELETE CASCADE** — удаление курса автоматически удалит все его задания.
- **ON DELETE SET NULL** — задания останутся, но `course_id` станет `NULL` (столбец должен это допускать).
- без указания (по умолчанию) — попытка удалить курс, у которого ещё есть задания, завершится ошибкой `FOREIGN KEY constraint failed`, пока задания не удалят/переназначат вручную.

Частые ошибки

- Забыть `PRAGMA foreign_keys = ON`; — без этого SQLite вообще не проверяет внешние ключи, и "битые" ссылки можно создать без единой ошибки.
- Перепутать, какая таблица родительская, а какая дочерняя — внешний ключ всегда живёт в дочерней таблице и указывает на первичный ключ родительской.
- Попытаться выразить связь "многие ко многим" одним `FOREIGN KEY` без промежуточной таблицы — так не работает, нужна отдельная таблица-связка.

Пример посложнее: несколько связанных таблиц сразу

В реальных системах внешние ключи редко идут по одной паре таблиц — обычно это целая сеть связей. Ниже — пример из системы управления пользователями, ролями, отделами и проектами: одна таблица может быть одновременно и родительской (для одних связей), и дочерней (для других).

Пример связей между таблицами

Основные ключи и внешние ключи

roles		
id	name	description
 1	admin	Полный доступ
2	teacher	Преподаватель
3	student	Студент

users	
id	name
 1	Иванов И.И.
2	Петров П.П.
3	Сидоров С.С.
4	Кузнецова А.

projects			
id	name	description	department_id
 1	Website Redesign	Обновление сайта	1
2	Mobile App	Мобильное приложение	1
3	Marketing Campaign	Рекламная кампания	3
4	Internal Tool	Внутренний инструмент	2